# LeetCode 448 — Find All Numbers Disappeared in an Array

**1. Problem Title & Link**

**Title:** LeetCode 448: Find All Numbers Disappeared in an Array

**Link:** https://leetcode.com/problems/find-all-numbers-disappeared-in-an-array/

**2. Problem Statement (Short Summary)**

Given an array nums of length n where:

$1 \le nums[i] \le n$

Some numbers appear twice and some numbers are missing.

Return all numbers in the range:

[1, n]

that do not appear in the array.

**3. Examples (Input → Output)**

**Example 1**

Input:

nums = [4,3,2,7,8,2,3,1]

Output:

[5,6]

Explanation:

Numbers from 1 to 8 are:

1 2 3 4 5 6 7 8

Present:

1 2 3 4 7 8

Missing:

5 6

**Example 2**

Input:

nums = [1,1]

Output:

[2]

**Example 3**

Input:

nums = [1,2,3,4]

Output:

[]

**4. Constraints**

n == nums.length

1 <= n <= 100000

1 <= nums[i] <= n

Follow-up:

Can you solve it without extra space?

This hint leads to the optimal solution.

**5. Core Concept (Pattern / Topic)**

**Array Marking**

**In-Place Hashing**

This is a famous interview pattern.

**6. Thought Process (Step-by-Step Explanation)**

**Brute Force Approach**

For every number from:

1 to n

Check whether it exists in the array.

Example:

Find 1

Find 2

Find 3

...

Time Complexity:

$O(n^2)$

Too slow.

**Better Approach (HashSet)**

Store all numbers in a set.

Then check:

1 to n

If missing from set:

add to answer

Time:

O(n)

Space:

O(n)

Works well.

**Optimal Thinking**

Given:

1 <= nums[i] <= n

Every value corresponds to an index.

Example:

Value 1 → Index 0

Value 2 → Index 1

Value 3 → Index 2

...

We can use the array itself as a marker.

**Key Idea**

Whenever we see:

value = x

Mark:

index = x - 1

as visited.

Use negative sign.

Example:

nums = [4,3,2,7,8,2,3,1]

See:

4

Mark:

index 3

negative.

See:

3

Mark:

index 2

negative.

Continue.

At the end:

Positive positions

=

Missing numbers

**7. Visual / Intuition Diagram (ASCII)**

```
Initial:
Index

0 1 2 3 4 5 6 7

Value

4 3 2 7 8 2 3 1
Visit 4
Mark index 3

4 3 2 -7 8 2 3 1
Visit 3
Mark index 2

4 3 -2 -7 8 2 3 1
Continue...
Final array:
-4 -3 -2 -7 8 2 -3 -1
Positive positions:
Index 4 → Number 5
Index 5 → Number 6
Answer:
[5,6]
```

**8. Pseudocode**

```
for each number

    index = abs(number) - 1

    make nums[index] negative

for each index

    if nums[index] > 0

        answer.add(index + 1)
```

```
return answer
```

### 9. Code Implementation

**Python**

```
class Solution:
    def findDisappearedNumbers(self, nums):

        for num in nums:

            index = abs(num) - 1

            nums[index] = -abs(nums[index])

        result = []

        for i in range(len(nums)):

            if nums[i] > 0:
                result.append(i + 1)

        return result
```

**Java**

```
import java.util.*;

class Solution {

    public List<Integer> findDisappearedNumbers(int[] nums) {

        for(int num : nums) {

            int index = Math.abs(num) - 1;

            nums[index] = -Math.abs(nums[index]);
        }

        List<Integer> result = new ArrayList<>();

        for(int i = 0; i < nums.length; i++) {

            if(nums[i] > 0) {

                result.add(i + 1);
            }
        }
```

```
        return result;
    }
}
```

**10. Time & Space Complexity**

**Time Complexity**

O(n)

Two linear scans.

**Space Complexity**

O(1)

Ignoring output array.

Uses the input array itself.

**11. Common Mistakes / Edge Cases**

**Mistake 1**

Forgetting abs()

Wrong:

int index = nums[i] - 1;

Correct:

int index = Math.abs(nums[i]) - 1;

**Mistake 2**

Using:

nums[index] *= -1;

Can accidentally turn negative back to positive.

Correct:

nums[index] = -Math.abs(nums[index]);

**Mistake 3**

Confusing:

index

with

value

**12. Detailed Dry Run**

Input:

nums = [4,3,2,7,8,2,3,1]

**First Pass**

| Value | Mark Index |
|---|---|
| 4 | 3 |
| 3 | 2 |
| 2 | 1 |
| 7 | 6 |
| 8 | 7 |
| 2 | 1 |
| 3 | 2 |
| 1 | 0 |

Array becomes:

[-4,-3,-2,-7,8,2,-3,-1]

**Second Pass**

Positive values:

Index 4 → Number 5

Index 5 → Number 6

Answer:

[5,6]

**13. Common Use Cases (Real-Life / Interview)**

**Presence Tracking**

Used for:

Attendance systems

Inventory tracking

Missing IDs

Transaction validation

Data integrity checks

Instead of extra memory, reuse existing storage.

**14. Common Traps (Important!)**

**Trap 1**

Not using absolute value.

**Trap 2**

Making an already-negative number positive again.

**Trap 3**

Forgetting:

Value x maps to index x-1

**Trap 4**

Assuming duplicates are unique.

Duplicates are expected.

**15. Builds To (Related LeetCode Problems)**

LC 448 Find Disappeared Numbers

↓

LC 41 First Missing Positive

↓

LC 287 Find Duplicate Number

↓

LC 442 Find All Duplicates

↓

LC 645 Set Mismatch

These are classic Array Marking / Cyclic Sort problems.

**16. Alternate Approaches + Comparison**

| Approach | Time | Space | Notes |
|---|---|---|---|
| Brute Force | $O(n^2)$ | O(1) | Too slow |
| HashSet | O(n) | O(n) | Easy |
| Boolean Array | O(n) | O(n) | Good |
| Array Marking | O(n) | O(1) | Optimal |

**Approach 1: HashSet**

Set<Integer> set = new HashSet<>();

Easy to understand.

**Approach 2: Boolean Array**

boolean[] seen = new boolean[n+1];

Common beginner solution.

**Approach 3: Array Marking**

Reuse the original array.

Best solution.

**17. Why This Solution Works (Short Intuition)**

Every number belongs to the range:

1..n

Therefore each value has a unique corresponding index.

By marking that index as negative, we record that the number exists.

Any index left positive represents a number that never appeared.

Hence:

index + 1

is missing.

**18. Variations / Follow-Up Questions**

**Follow-Up 1**

Find duplicate numbers.

Leads to:

LC 442

**Follow-Up 2**

Find the first missing positive.

Leads to:

LC 41

**Follow-Up 3**

Find exactly one duplicate.

Leads to:

LC 287

**Follow-Up 4**

Can you solve using cyclic sort?

Yes.

Place every number at its correct position:

value x → index x-1

Then scan for mismatches.

**Pattern Learned**

Array Marking

Value
↓
Index Mapping
↓
Mark Visited
↓
Unvisited Positions
↓
Missing Numbers